NEOLABS

SECURITY LABS | GREY-BOX PENTEST

Week 02 - The Ghost Login

Weekly Learning Pack | Learn + Connect + Operate

Programme: NeoLabs x Renaissance Innovation Labs Cybersecurity Internship
Week 02
Version: 1.0
Classification: Student Training Material - Authorised Synthetic Use Only
Authorised synthetic training use only.

Why this week matters

Authentication is the boundary between an anonymous user and an identified user. A grey-box tester studies that boundary with some legitimate internal knowledge, synthetic test accounts and a written scope. The goal is not to 'break into everything.' The goal is to verify whether the prepared authentication behaviour is secure, document what is reproducible and stop at the approved proof threshold.

Week 2 teaches a disciplined sequence: **authenticate to the lab, confirm scope, map normal behaviour, observe the prepared weakness, capture minimum proof, recommend a fix and retest.**

Learning outcomes

By the end of this pack you should be able to:

- explain authentication, authorisation and session management at a beginner level;
- use the NeoLabs broker to retrieve the current authorised pod targets;
- understand why a stable base URL can coexist with changing lab target IPs;
- use a bounded Nmap discovery profile when the assignment publishes an IP/CIDR;
- map a login flow with a browser and Burp Suite Community;
- compare expected and unexpected authentication responses without broad password guessing;
- capture useful request/response evidence while protecting credentials;
- write a concise authentication finding;
- retest a fixed scenario safely.

1. Scope comes before tools

Before opening Burp or Nmap, authenticate to the NeoLabs lab broker:

```
python3 tools/neolabs.py login
python3 tools/neolabs.py connect
python3 tools/neolabs.py scope
python3 tools/neolabs.py targets
```

The **lab base URL** is the stable authentication/discovery entry point. It may not be the target you test.

A live manifest may publish:

- one exact application hostname;
- one exact lab IP;
- several approved service endpoints;
- a small approved CIDR for a host-discovery exercise;
- port hints when the week's task requires them.

The manifest is generated at runtime and is not committed to this public repository.

> **Scope boundary:** A target is authorised only when it is returned by the current broker manifest and permitted by the written Week 2 GitHub Issue. Reachability alone is never permission.

2. Authentication vs authorisation vs session

Authentication answers: 'Who are you?'

Authorisation answers: 'What are you allowed to access?'

Session management answers: 'How does the application remember that authenticated state across requests?'

The Ghost Login scenario focuses mainly on authentication behaviour, but testers should notice where the three concepts meet.

3. Establish normal behaviour first

Use only the synthetic accounts supplied for the scenario. Record what a normal login looks like before you investigate anything unusual.

Observe:

- login page/request endpoint;
- request method;
- normal required fields;
- response status and redirect behaviour;
- visible success/failure message;
- whether a session identifier/cookie appears after success;
- whether logout changes the session state;
- whether an obviously incorrect synthetic password produces a consistent failure response.

Do not collect other users' credentials and do not attempt broad password guessing.

4. Using Burp Suite Community safely

For this week, Burp is mainly an observation and controlled replay tool.

Recommended workflow:

1. Configure the browser proxy.
2. Keep the Target scope limited to the hostname returned by `neolabs targets`.
3. Perform one normal synthetic login through the browser.
4. Review the request and response in HTTP history.
5. Send only the relevant request to Repeater when the assignment explicitly requires controlled comparison.
6. Change only the specific field/condition required by the exercise.
7. Record the result and stop when the proof threshold is met.

Do not enable broad automated scanning against the lab unless a later assignment explicitly authorises it.

> **Evidence requirement:** Redact passwords, Access Codes, bearer tokens and session secrets. Evidence should show the relevant behaviour without exposing reusable credentials.

5. Nmap when the week requires an IP

Some Grey-Box weeks need network/service discovery. The toolkit intentionally supports this without giving students a permanent hard-coded lab IP.

First read the current scope:

```
python3 tools/neolabs.py scope
python3 tools/neolabs.py targets
```

Then use the fixed wrapper with exactly one returned target:

```
bash scripts/safe-nmap.sh 10.40.3.21
```

If - and only if - the broker explicitly returns a CIDR:

```
bash scripts/safe-nmap.sh 10.40.3.16/28
```

The validator reads `runtime/access-manifest.json` and rejects unlisted/out-of-range hosts. The wrapper uses a bounded low-rate profile and does not accept arbitrary additional Nmap flags.

Week 2 may not require Nmap for every pod. If the GitHub Issue does not ask for discovery, do not scan just because the command exists.

6. What counts as an authentication weakness?

A tester looks for behaviour that violates the application's intended authentication rules. In this controlled scenario, the mentor has prepared a narrow weakness or unsafe condition for students to identify.

Your job is to answer:

- What should the application have required?
- What actually happened?
- Under what synthetic, authorised conditions can it be reproduced?
- What security impact does that difference create inside the lab story?
- What is the minimum evidence needed to prove it?

Avoid conclusions that go beyond the evidence. A strange response is not automatically an authentication bypass.

7. Request/response evidence

A useful evidence record contains:

Item	Record
Time	When the controlled test occurred
Target	Server-returned authorised hostname/IP
Synthetic account	Redacted label, not a password
Request summary	Method, path and relevant non-secret field
Response summary	Status, redirect/message and relevant state change
Expected behaviour	What should have happened
Observed behaviour	What actually happened
Evidence ID	Screenshot/export reference

Do not paste a full live session token into your report.

8. Writing the finding

A beginner finding should be reproducible and calm.

Title

Name the condition, not the drama.

Summary

One paragraph explaining the unsafe authentication behaviour.

Preconditions

State the synthetic account/state needed.

Reproduction

List the smallest approved sequence another mentor can repeat.

Evidence

Reference screenshots/request observations with secrets redacted.

Impact

Explain what the condition could allow inside the lab scenario.

Remediation

Describe the security property that should be enforced. Avoid prescribing a fragile one-line patch when you do not know the full application design.

Retest

State whether the fixed branch prevents the original condition while normal login still works.

9. Proof threshold and stop conditions

Stop when:

- the prepared behaviour is proven with the required synthetic evidence;
- continuing would only produce more copies of the same proof;
- the application becomes unstable;
- an unassigned host/pod appears reachable;
- real personal data appears;
- the next action would require persistence, destructive change or broad automation;
- the written scope is unclear.

A professional tester knows when enough evidence is enough.

10. Retesting the fixed scenario

Retesting should answer two questions:

1. Does the original unsafe behaviour still reproduce?
2. Does the legitimate authentication workflow still function?

Keep the same test preconditions when possible so the before/after comparison is meaningful. Record the fixed-version evidence separately.

Week 2 operating sequence

```
1. Read the GitHub Issue and Rules of Engagement.
2. Study this learning pack.
3. Run neolabs login and neolabs connect.
4. Run neolabs scope and neolabs targets.
5. Baseline normal login behaviour using synthetic credentials.
6. Perform only the controlled comparison required by the assignment.
7. Capture minimum proof and stop.
8. Write the finding and remediation recommendation.
9. Retest the fixed version.
10. Submit through the central assignment repository Pull Request workflow.
```

Quick knowledge check

1. Why is `NEOLABS_LAB_BASE_URL` not automatically the pentest target?
2. When may you scan a CIDR with the NeoLabs Nmap wrapper?
3. What is the difference between authentication and authorisation?
4. Why should passwords and session tokens be redacted from evidence?
5. What two questions should a retest answer?

Remember

Scope first. Baseline normal behaviour. Prove the minimum. Stop at the threshold.